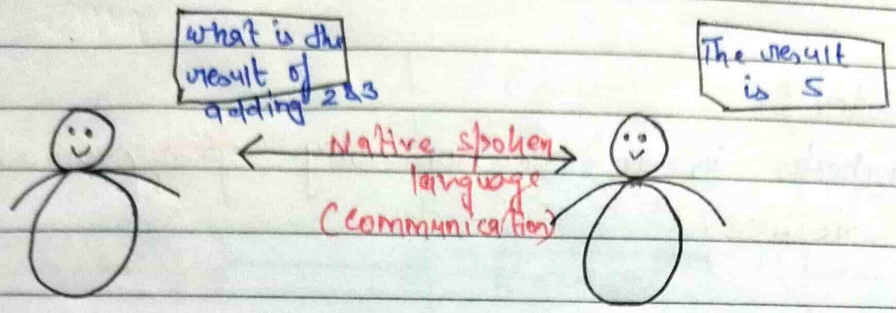
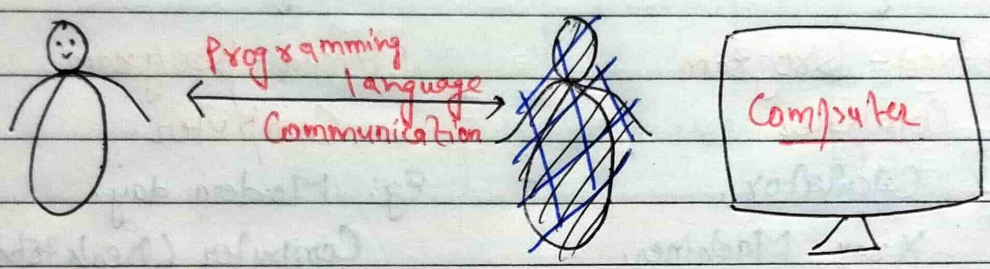
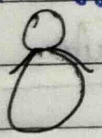
JAVA





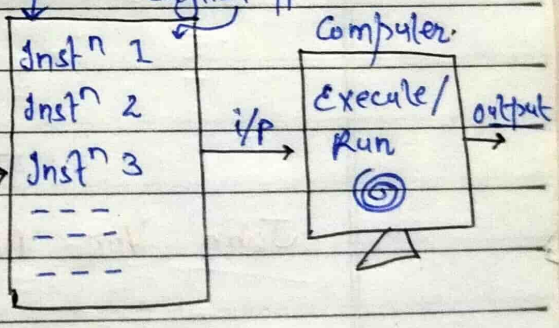
Information technology (IT) Industry



Programming language
C, C++, C#, Java, python, Javascript, etc.

PL (programming language)
(Grammar) (syntax)

English alphabet



Program
(set of inst^n)

↓

software/ application (App)
(set of program)

Program:- A set of inst^n that is given as input to the computer to perform a certain task.

Akash

Software/Application → It is a set of many programs.

MON TUE WED THU FRI SAT SUN

Page No.: _____
Date: ___/___/_____

→ Computer :-
A computer is an electronically programable device.

Computer :-

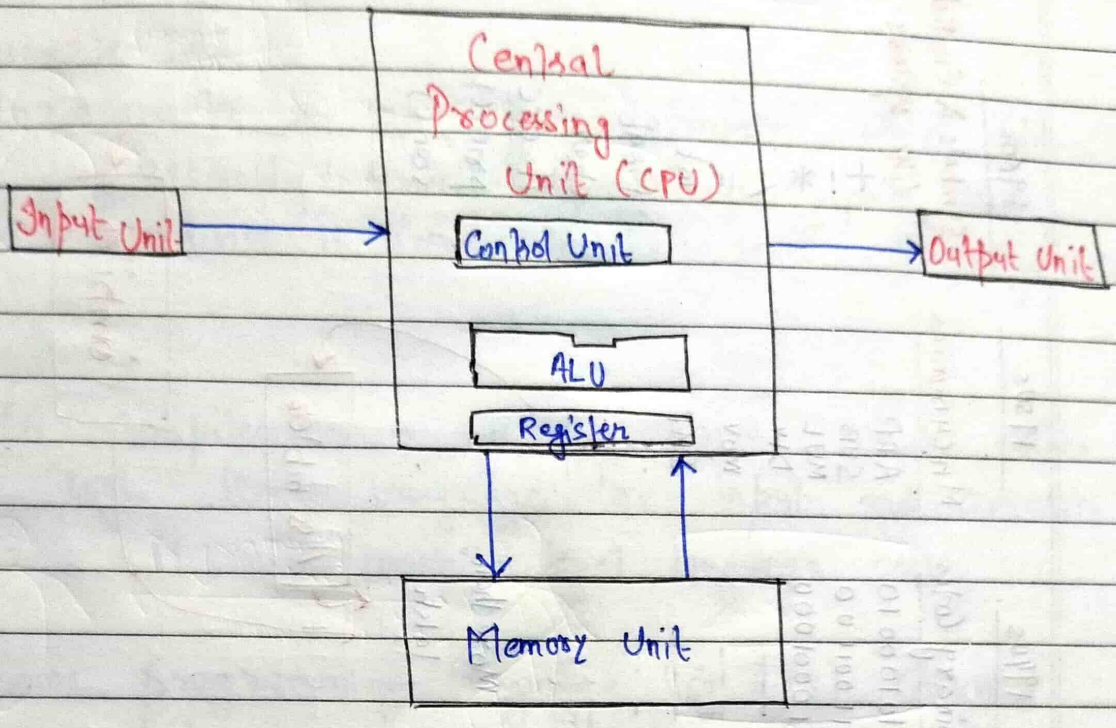
Fixed-program Computer
Eg; Calculator, X ray Machines, Washing machines, etc.

Stored-program Computer
Eg; Modern day Computer (Desktops, PCs, laptops, mobiles, etc).

ORganization of a Computer.

John Von Neumann Architecture / Model :

1. Input size
2. Processing Unit
3. Memory Unit
4. Output Unit



1940s

Conductor & Insulator

Semiconductor

1940s

Machine level language (MLL) or low level language (LLL)

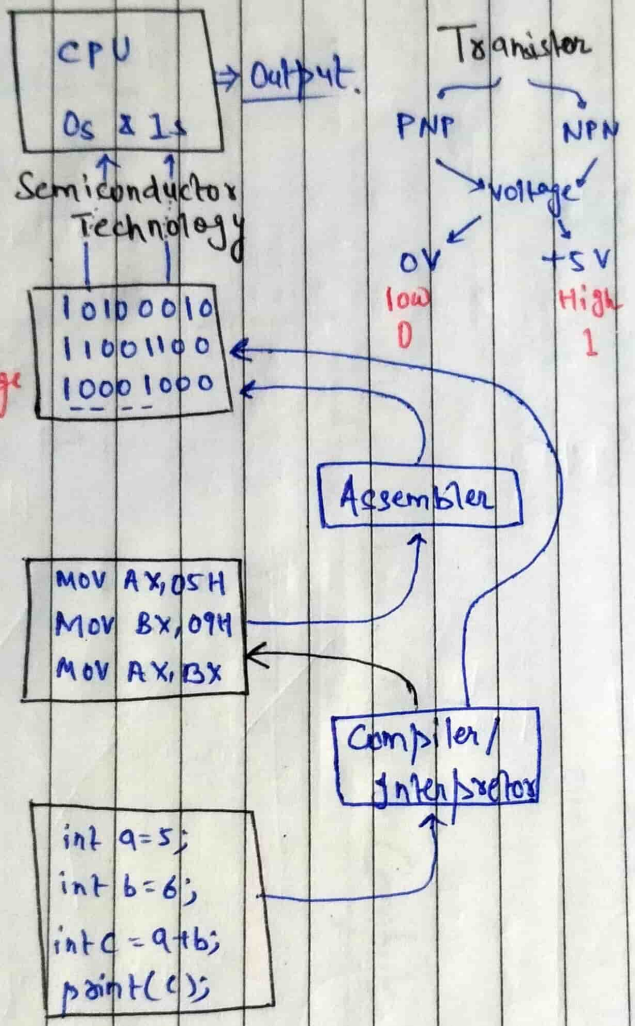
1950s

Assembly level language (ALL)

1960s

High level language (HLL)

(C, C++, C#, Python, Java, JS, etc)



1940s

Binary Codes

10100010
11001100
10001000

.
.
.

More than lakh

1950s

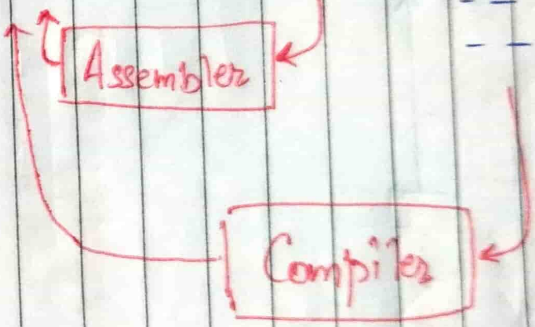
Mnemonics

ADD
SUB
MUL
Div
MOV
JMP
--
--
--
.

1960s

Symbols & English like syntax

+
-
*
/
=
int
float
char
if-else
print()
scan()
--
--



Akash

MON TUE WED THU FRI SAT SUN

Page No.:
Date: __/__/____

Assembler :-
An assembler is an system software which takes assembly level language as input and converts it into machine level language code.

Compiler :-
A Compiler is a system software which takes high level language as input and converts it into machine level language code.

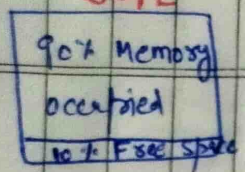
- Some programming language (Eg C++) Converts high level language into assembly level language then into low level language code.
  It depends upon compiler.

Programming Paradigms :-

1. Unstructured / Non-Structured Style.
2. Structured / procedure Style.
3. Object-oriented programming Style.

Popular High-level language.

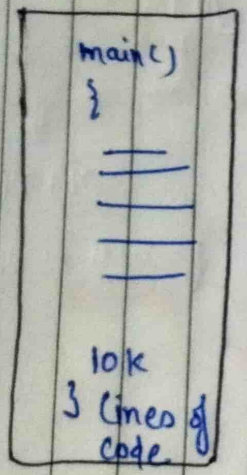
BCPL



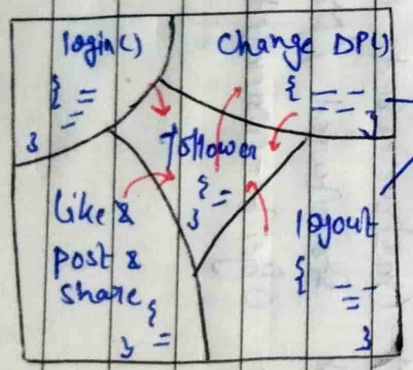
B

1972
Dennis Ritchie
Bell labs, USA
C
Structured-oriented PL

FORTAN
Unstructured



Small & simple task



Function/ Procedure/ Sub-programs/ Modules--

large & complex Task

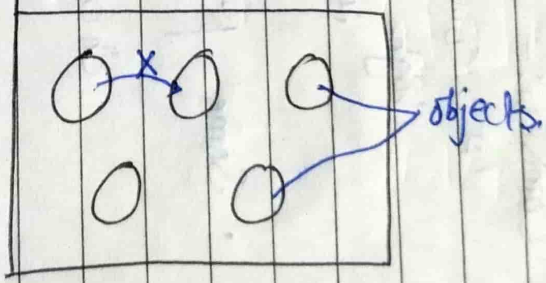
- function ↑
- Data ↓

1979
Bjarne Stroustrup
Bell labs, USA

C(POP) Simula(OOP)

"C with Class"

1985 - C++ Hybrid



- Data ↑
- function ↓

1990s

"Internet" - WWW

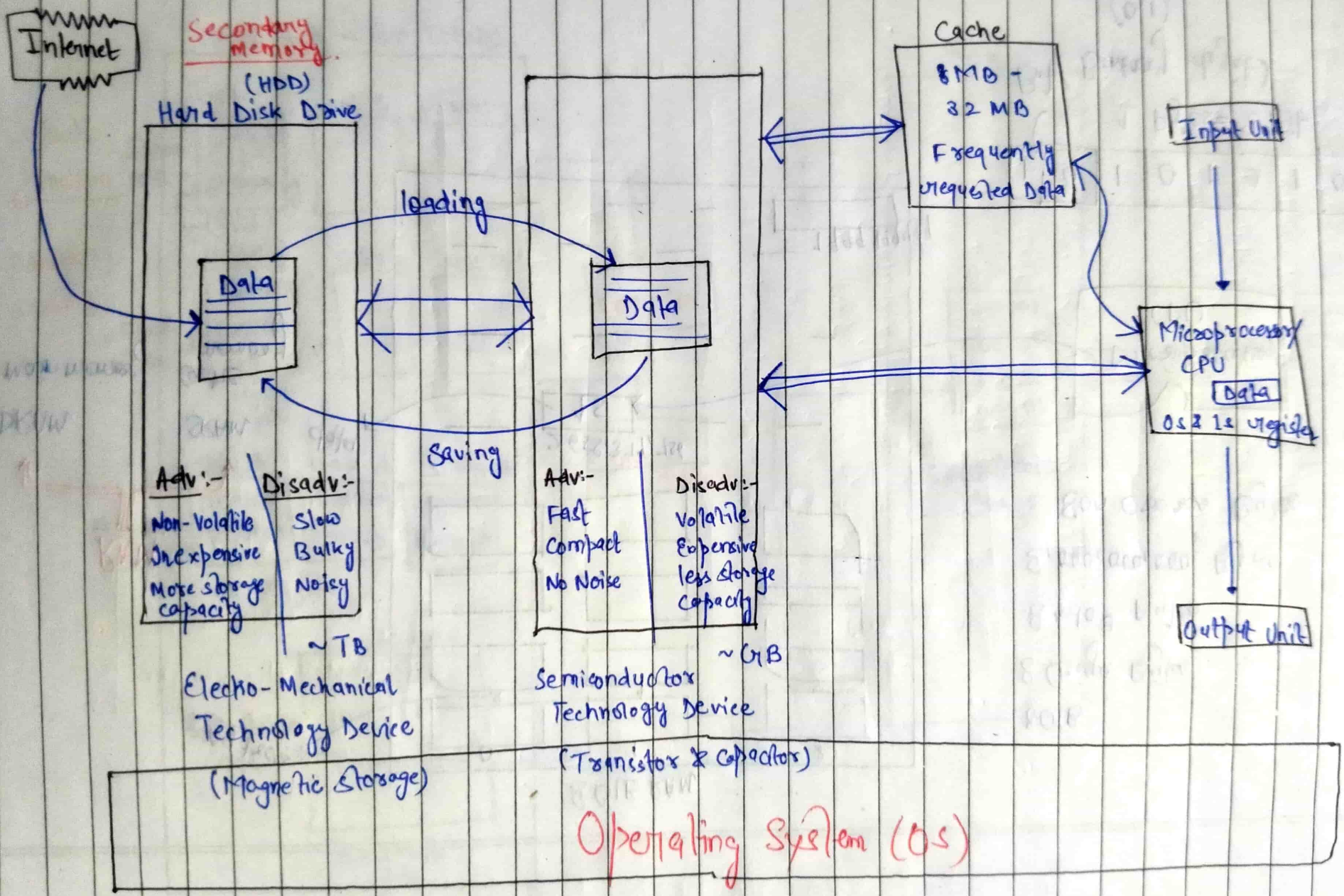
Sun Microsystem, Inc.
California, USA

James Gosling
11 member teams
green team

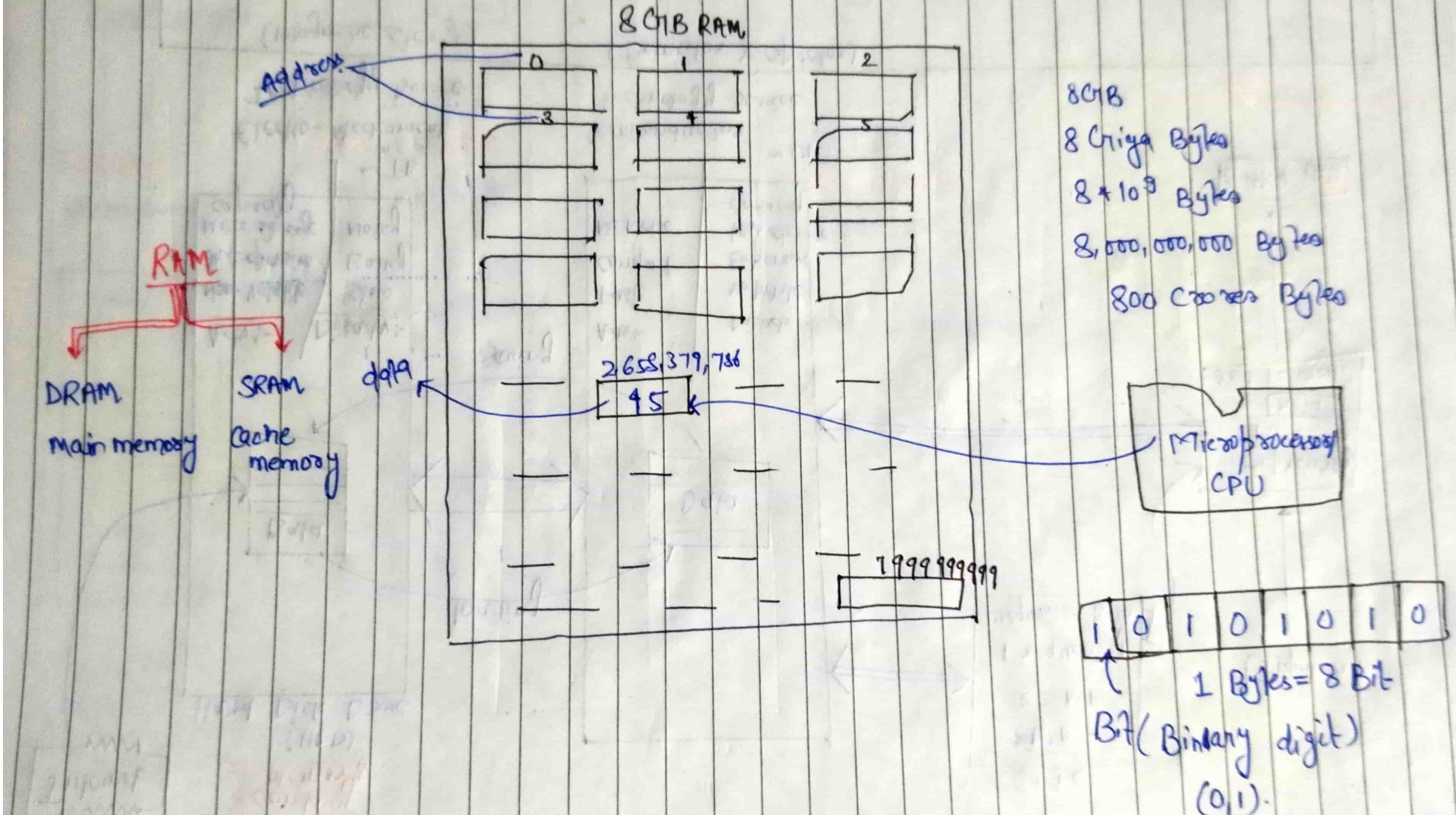
TV sets / Electronic Devices + Internet
"Platform independent"

1995 - JAVA

Akash

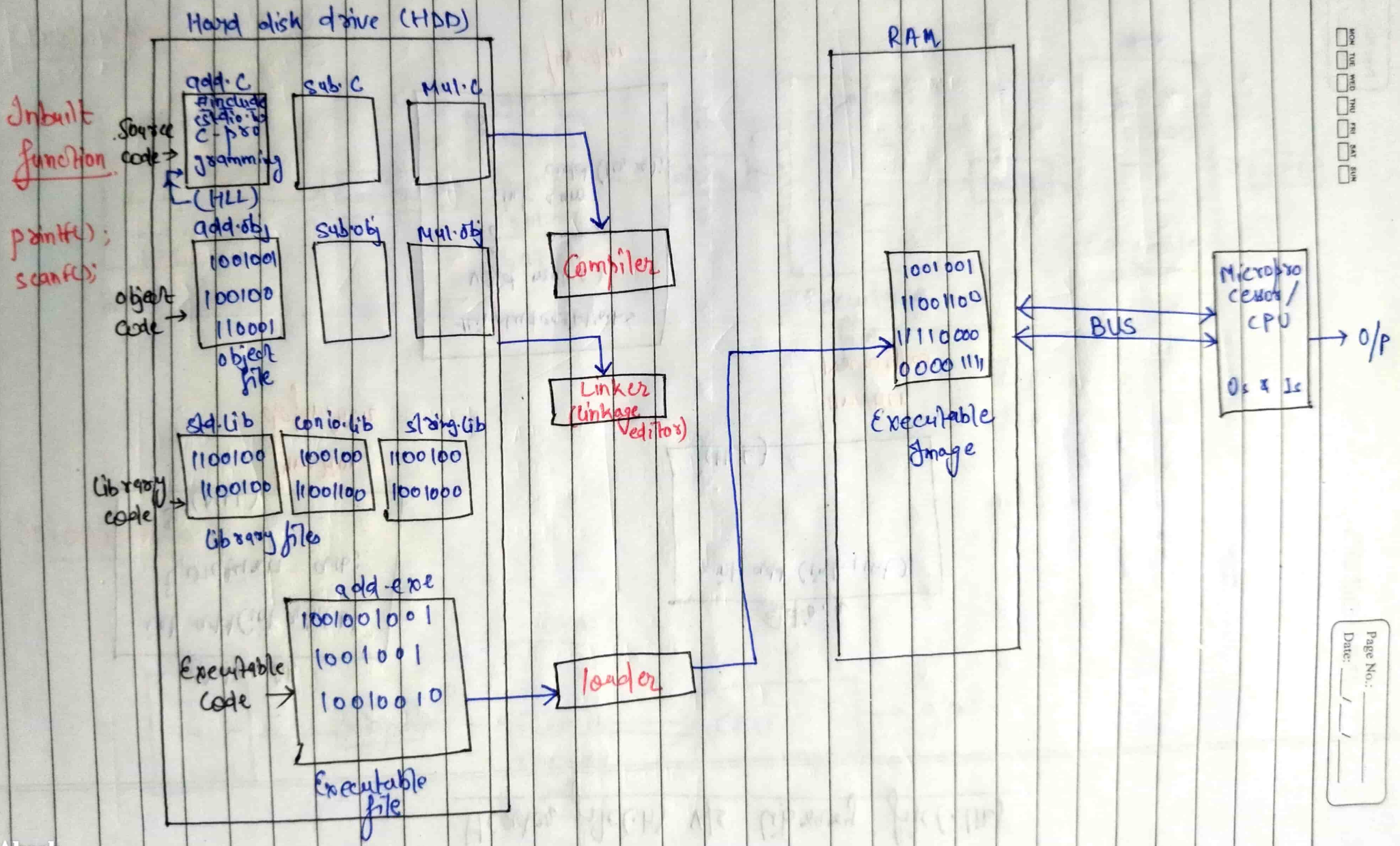

Internet
Secondary Memory
(HDD)
Hard Disk Drive
Data
loading
saving
Data
Cache
8MB - 32 MB
Frequently requested Data
Input Unit
Microprocessor
CPU
Data
Output Unit
Adv:-
Non-Volatile
Inexpensive
More storage capacity
Disadv:-
Slow
Bulky
Noisy
~TB
Electro-Mechanical Technology Device
(Magnetic Storage)
Adv:-
Fast
Compact
No Noise
Disadv:-
Volatile
Expensive
less storage capacity
~GB
Semiconductor Technology Device
(Transistor & Capacitor)
Operating System (OS)

Akash


8 GB RAM
Address
0
1
2
3
4
5
RAM
DRAM
main memory
SRAM
cache memory
data
2,658,379,736
45
1999999999
Microprocessor
CPU
8GB
8 Giga Bytes
8 * 10^9 Bytes
8,000,000,000 Bytes
800 Crores Bytes
1 0 1 0 1 0 1 0
1 Bytes = 8 Bit
Bit (Binary digit)
(0,1).

Akash

{ compiler, Linker, loader → software } All are



Akash

Header file(.h) v/s Library file(.lib)

stdio.lib

```
int add(int a, int b)
{ return a+b;
}
(HLL)
```

function definition

stdio.h

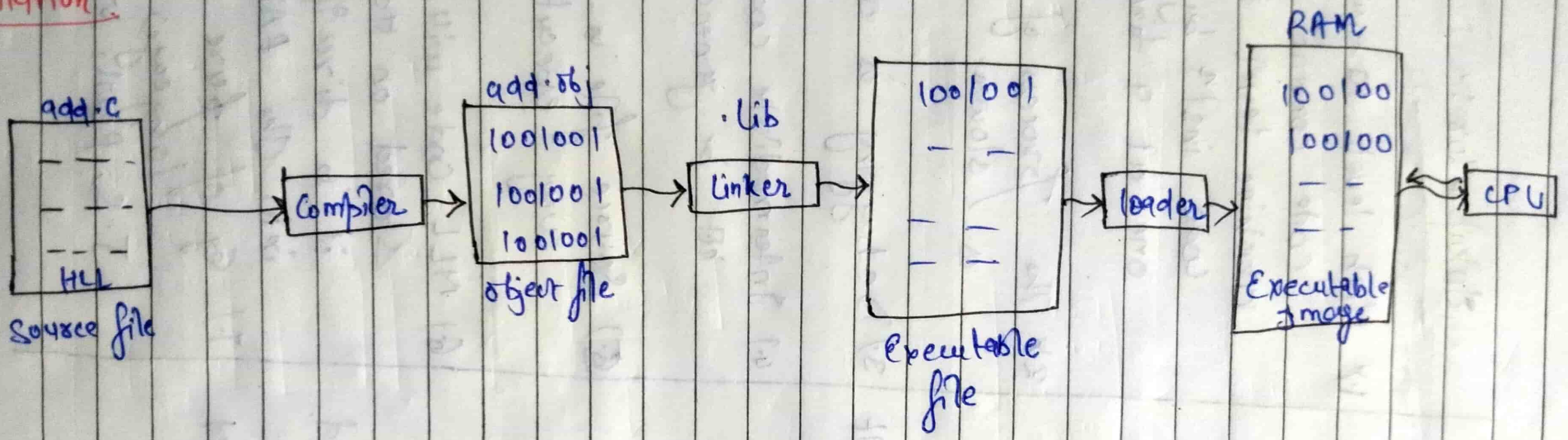
```
int add (int, int)

(HLL)
```

function prototypes

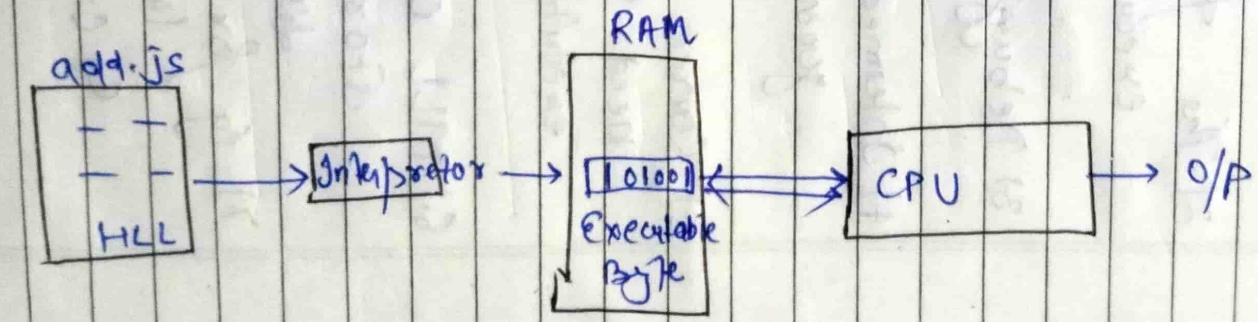
```
#include<stdio.h>
void main()
{
  int sum =
      add (10, 20);
}
```

function call

Compilation :-



Interpretation :-



Akash

Compiler :~	Interpretor :~
1.) Entire high level language code convert into machine level language code in one shot	1.) High level language code convert into machine level language code instⁿ by instⁿ one at a time.
2.) The process of execution is faster	2.) The process of execution is slower.
3.) Debugging is difficult	3.) Debugging is easy.
4.) Intermediate code is generated	4.) Intermediate code will not be generated.
5.) Source file is not need for multiple execution.	5.) Source file is needed for every execution.
6.) MLL code will be stored on Hard disk.	6.) MLL code will not be stored on Hard disk, it is directly loaded into the RAM.
7.) Eg for pure compiled language :- Eg, C++, C#, etc.	7.) Eg of pure interpreted language; J.S, Pearl, Ruby, etc.

Akash

MON TUE WED THU FRI SAT SUN

Page No.:
Date: ___/___/____

Java is hybrid programming language as it is both compiled as well as interpreted.

→ Platform independent.

HW Engineering:-

Platform

(Hardware) HW + SW (Software)

(Microprocessor) MP + OP (Operating system)

Eg;

intel i9 + Window 10

AMD Ryzen 9 + Linux

SW Engineering:-

Platform

↓

SW

O/S (Operating System)

Eg;

Window 10

Linux

Mac OS

In Simple term,

Platforms means Operating System

→ Platform of a computer refers to the combination of hardware & software components of a ~~comp~~ computer.

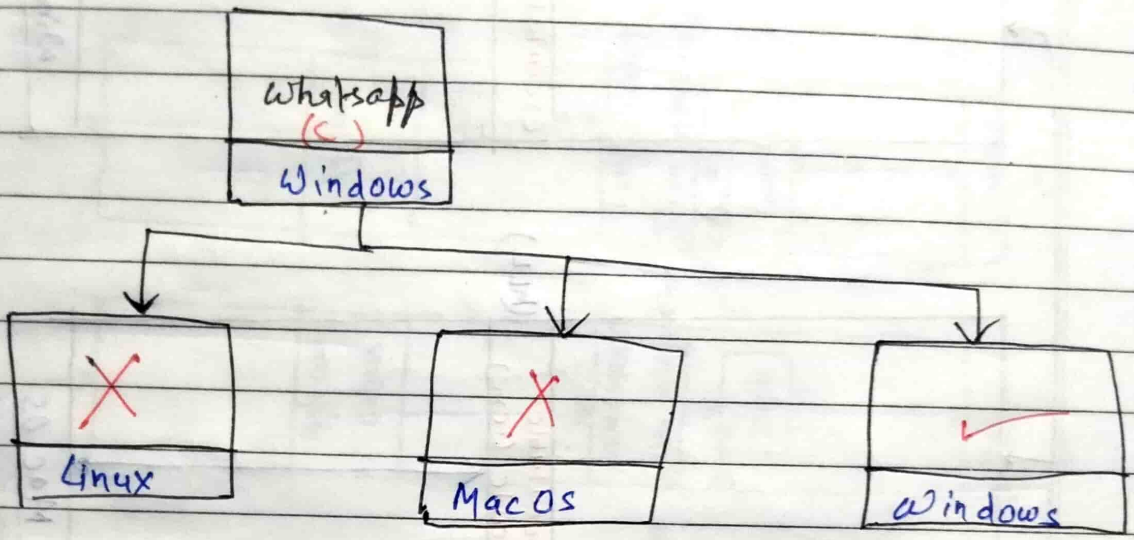
From Software Engg. perspective, the platform of a computer simple refers to the underline operating system of a computer.

Akash

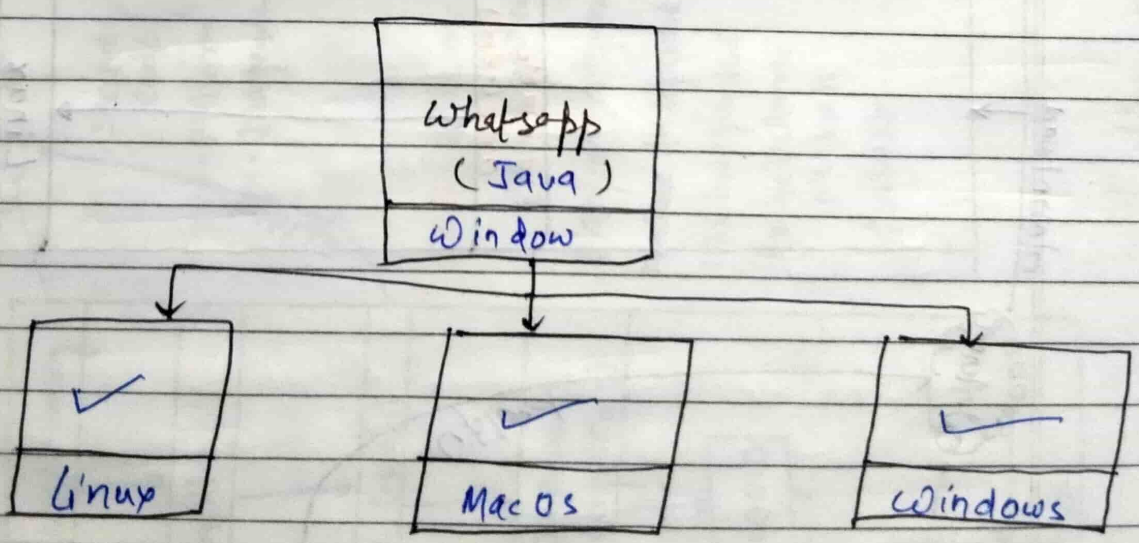
MON TUE WED THU FRI SAT SUN

Page No.:
Date: __/__/____

Platform Dependency :-

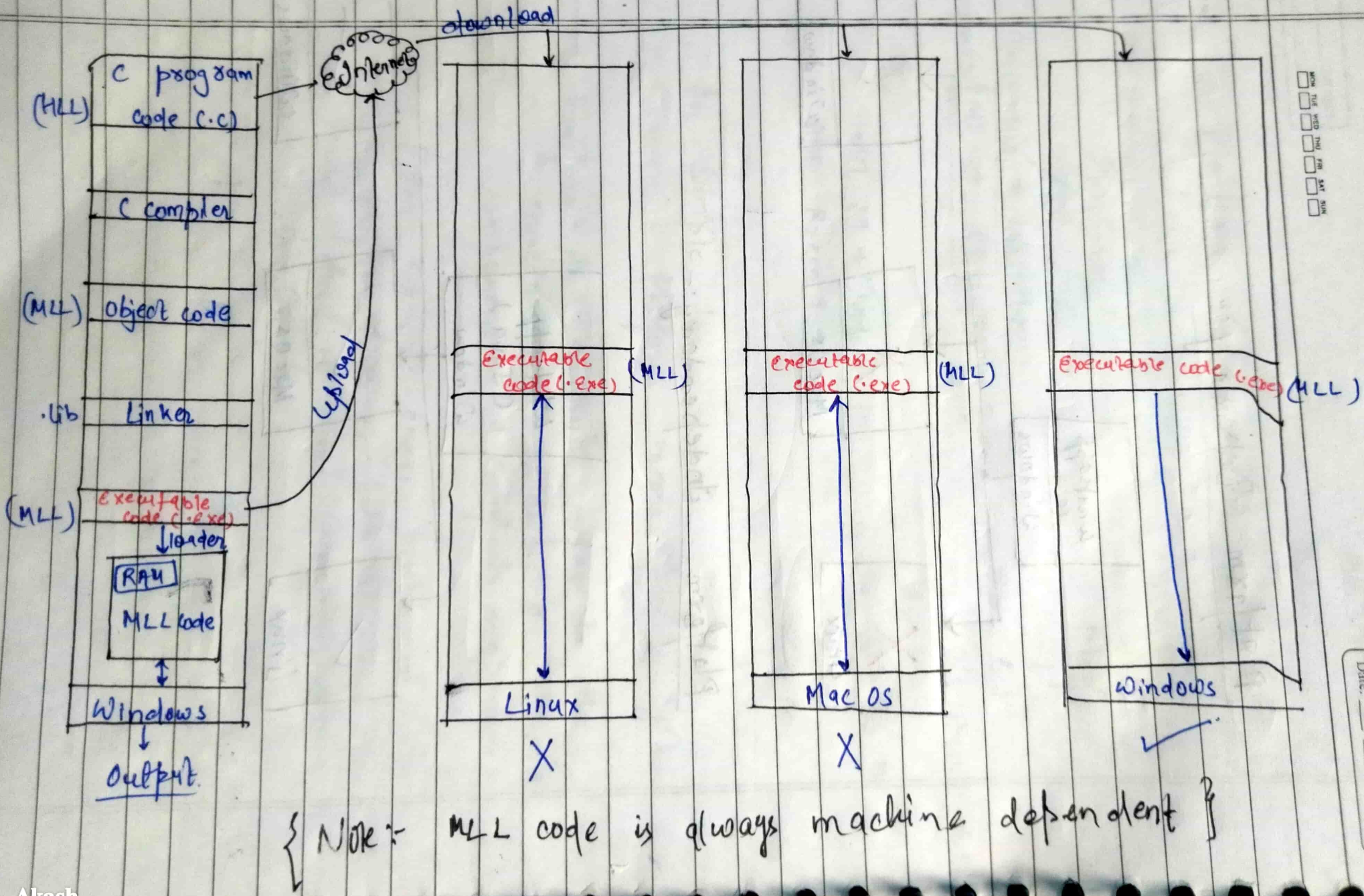


Platform Independency :-



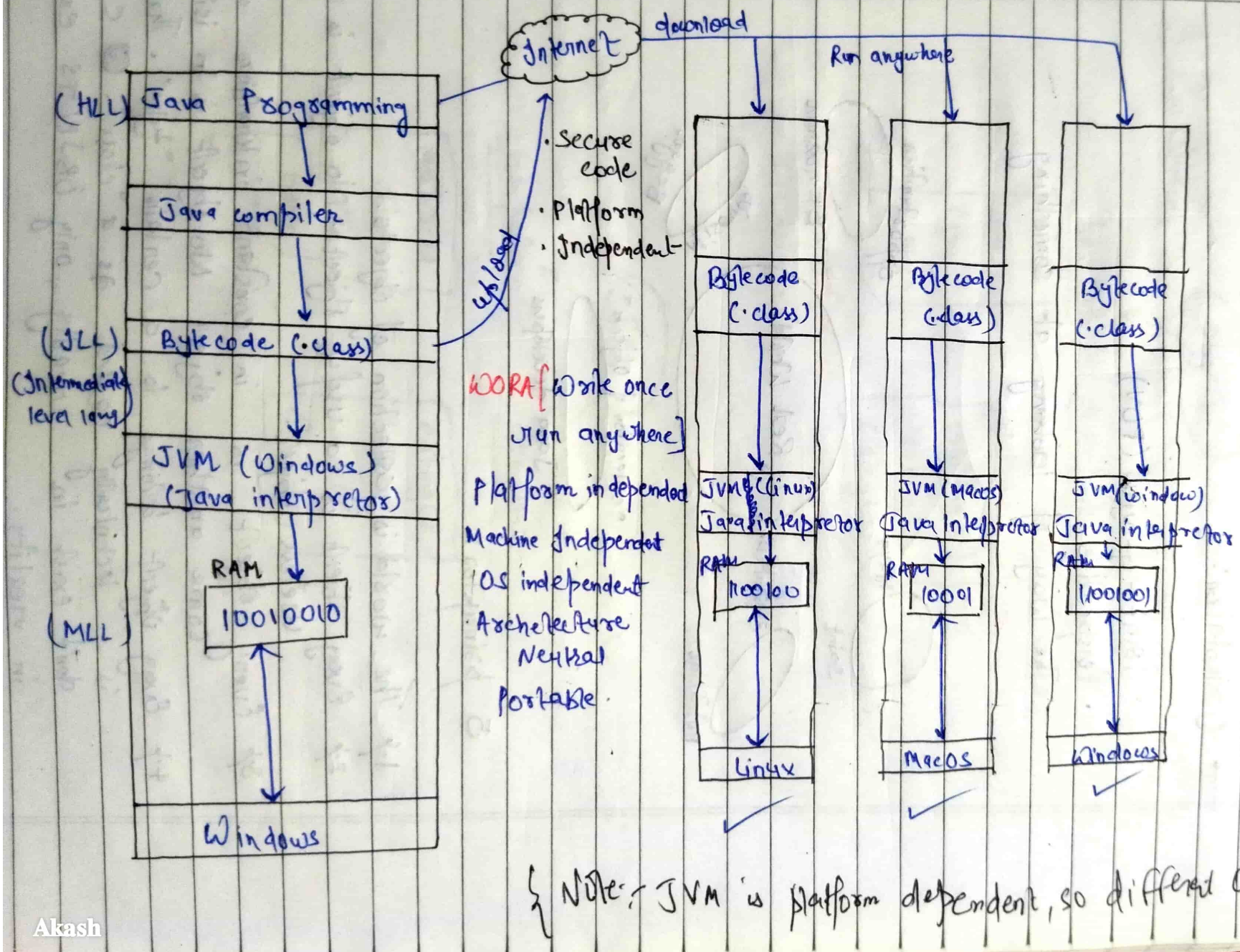
Akash

How C is platform dependency



{ Note :- MLL code is always machine dependent }

Akash

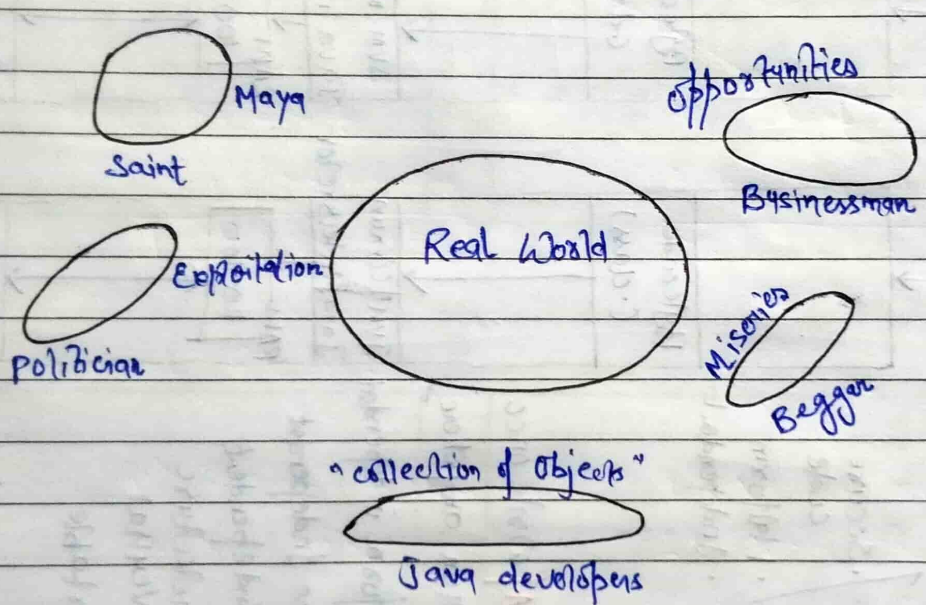

Internet
download
Run anywhere
(HLL) Java Programming
Java compiler
(JLL) Bytecode (.class)
(Intermediate level lang)
JVM (Windows)
(Java interpretor)
RAM
10010010
(MLL)
Windows
upload
· Secure code
· Platform
· Independent
WORA [Write once
Run anywhere]
Platform independent
Machine Independent
OS independent
Architecture
Neutral
Portable
Bytecode (.class)
JVM (Linux)
Java interpretor
RAM
100100
Linux
Bytecode (.class)
JVM (MacOS)
Java interpretor
RAM
10001
MacOS
Bytecode (.class)
JVM (window)
Java interpretor
RAM
1100100
Windows
{ Note: JVM is platform dependent, so different OS hap different JVM }

Akash

→ Object - Orientation :-

→ Orientation :-

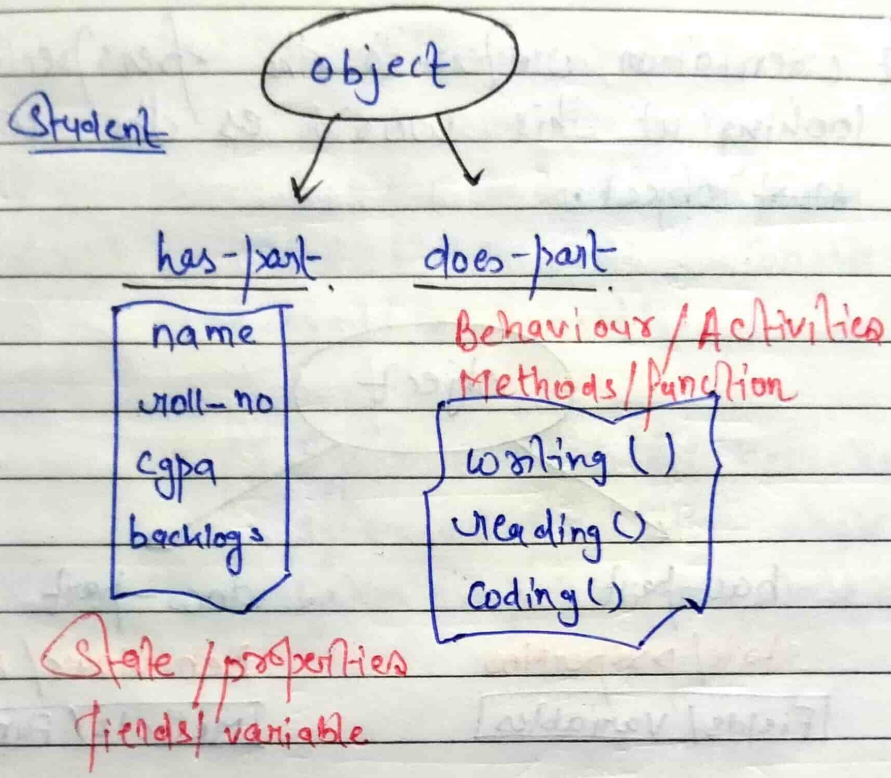
- Point - of - view (POV)
- Perspective
- The way of looking at something

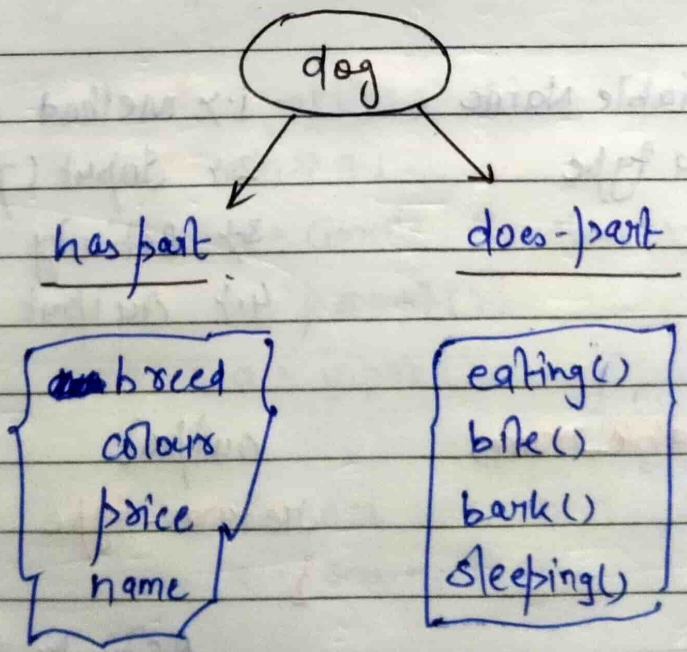


5 principles :

1. The world is collection of objects.
2. Every object is a useful object. No object is the useless object.
3. Every object is in constant interaction with some another object. No object is in isolation.
4. Every object belongs to a certain "type". That type is technically called as a "class". CLASSES don't exist in reality, only OBJECTS exist in reality.

5.) Every object has two parts:



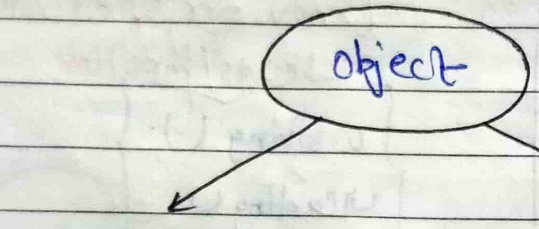


MON TUE WED THU FRI SAT SUN

Date: __/__/____

→ Object-Orientation:-

Object orientation refers to the perspective of looking at this world as the collection of ~~most~~ object.



General Syntax:

1.) Variable Name
2.) Data type

data_type variable_name;

1.) Method name
2.) Input (parameter)
3.) Activity (Body)
4.) Output (return type)

output input
return_type method_name (parameter)
{
Activity/task
// Body of the method
}

Akash

MON TUE WED THU FRI SAT SUN

Page No.:
Date: __/__/____

→ Printing Output :~

C → Printf("Hello World");
C++ → Cout("Hello World");
C# → Console.WriteLine("Hello World");
JavaScript → ~~Print("Hello World");~~ Console.log("Hello World");
Python → Print("Hello World");

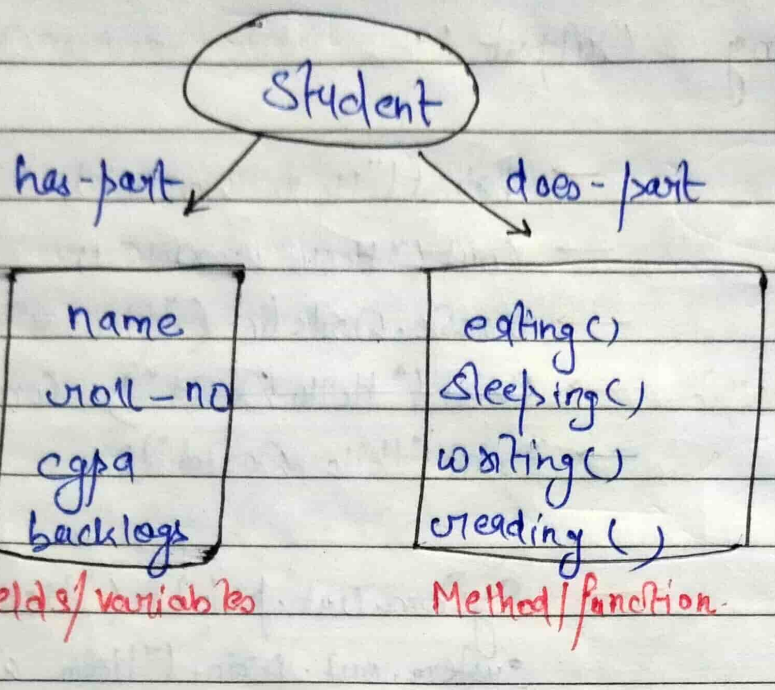
Java → System.Out.println("Hello World");
System.out.print("Hello World");
System.out.printf("Hello World");

→ Scanning Input :-

C → Scanf("%s", a);
C++ → cin >> a;
C# → a = Console.Readline();
Javascript → a = prompt();
Python → a = input();

Java → Scanner scan = new Scanner(System.in);
a = scan.next();

Akash



datatype variable_name

return_type method_name (parameter)
{
// body
}

```
Class Student
{
    String name;
    int roll_no;
    float cgpa;
    boolean backlogs;

    void eating()
    {
        System.out.println("Student is eating...");
    }

    void sleeping()
    {
        System.out.println("Student is sleeping...");
    }

    void writing()
    {
        System.out.println("Student is writing...");
    }
}
```

Has part

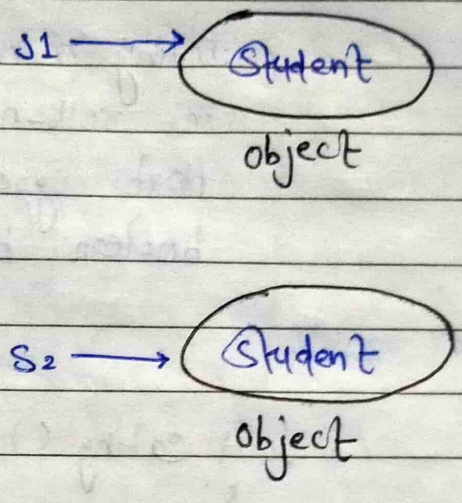
Does part

MON TUE WED THU FRI SAT SUN

Page No.:
Date: __/__/____

JVM

```
Student s1 = new Student();
Student s2 = new Student();

s1.eating();
s2.sleeping();
```

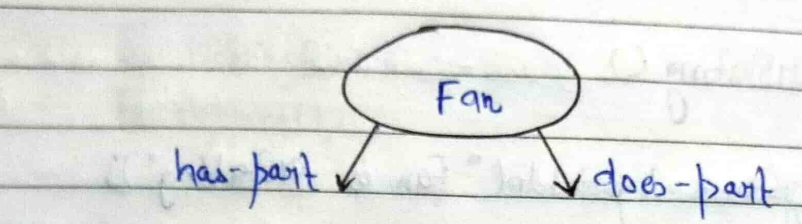


Akash

JAVA is Case-Sensitive

MON TUE WED THU FRI SAT SUN

Page No.:
Date: / /



brand
no_of_blades
price

starting()
rotating()
blowing Air()
stopping()

f_1	f_2	f_3
starting()	starting()	starting()
stopping()	rotating()	rotating()
		blowing Air()
		stopping()

```
Class fan
{
    String brand;
    string color;
    int no_of_blade;
    float price;

void starting()
{
    System.out.println("Fan is starting");
}
```

Akash

```
void rotating()
{
    system.out.println("Fan is rotating");
}

void blowingAir()
{
    system.out.println("Fan is BlowingAir");
}

void stopping()
{
    system.out.println("Fan is Stopping");
}

}

fan f1 = new fan();
fan f2 = new fan();
fan f3 = new fan();

f1.starting();
f1.stopping();

f2.starting();
f2.rotating();

f3.starting();
f3.rotating();
```

Akash

MON TUE WED THU FRI SAT SUN

Page No.:
Date: __/__/____

f3. blowingAir();
f3. stopping();

MON TUE WED THU FRI SAT SUN

Date: __/__/____

Java is Case-sensitive Programming Language

→ Naming Conventions :-

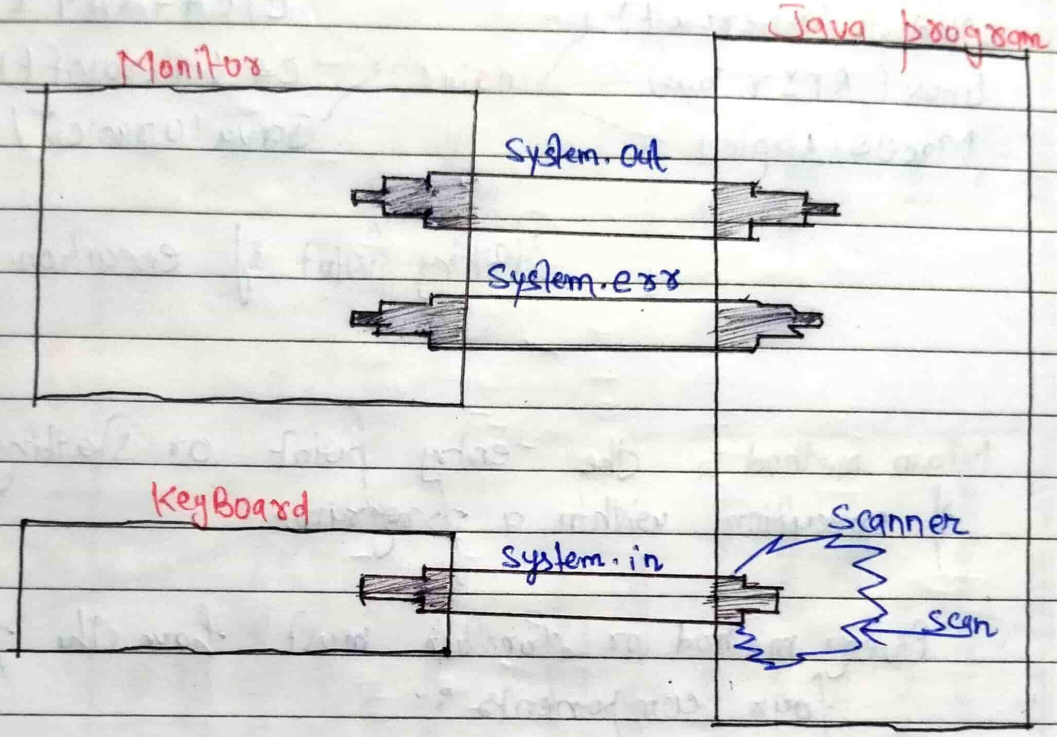
1. PascalCase Convention
2. camelCase Convention
3. kebab-case Convention
4. snake_case Convention

Naming Convention in Java :-

1. ClassName → PascalCase Convention - Noun
   - Fan
   - ElectricFan
2. variableName → camelCase Convention - Noun
   - brand
   - numberOfBlades
3. methodName → camelCase Convention
   - start()
   - blowingAir()
4. InterfaceName - PascalCase Convention
   - Runnable
   - Callable
   - Countable

Akash

→ Input - Output (I/O) in Java :-



Output :-

To display output messages:

System.out.println("Welcome to [redacted] my House");

To display errors messages:

system.[redacted]err.println("Invalid Input");

[redacted]

Scanner scan = new Scanner(System.in);

Akash

Main method in JAVA

Operating System	Programming languages
Windows (Microsoft)	C / C++ (AT & T labs)
Linux (AT & T labs)	C# (microsoft)
MacOS (Apple)	Java (Oracle) / Python (PSF)

main()

↓

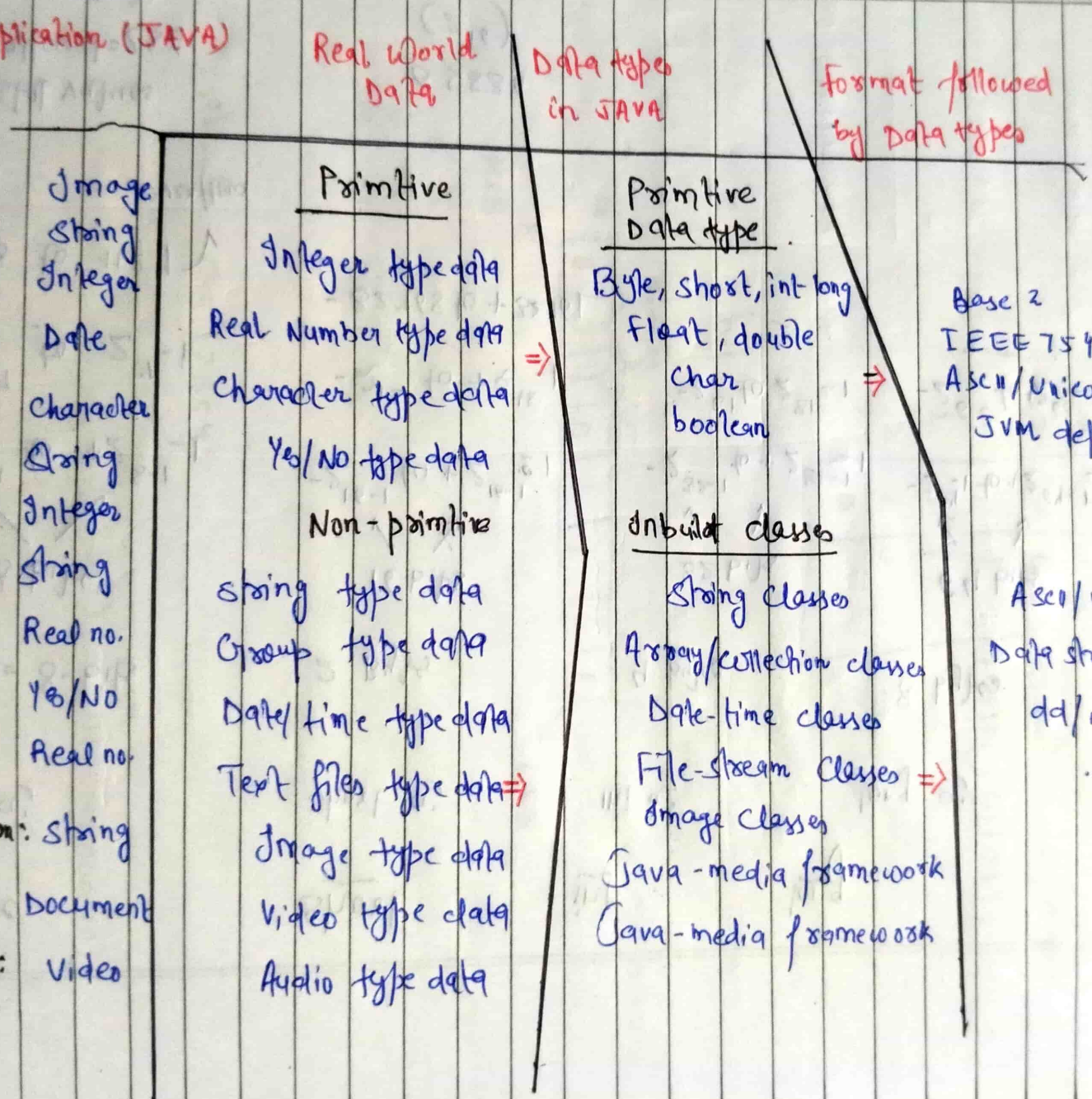
Starting point of execution.

→ Main method is the entry point or Starting point of execution within a program.

Every method or function must have the following four components :-

1. Name
2. Input
3. Activity / task
4. Output.

Data types in JAVA :-



→ Integer data type :-

Base - 2 Format

n bits

-2^{n-1} to $+2^{n-1}-1$

Byte

byte a;

1 byte = 8-bits

8 bits

-2^{8-1} to $+2^{8-1}-1$

-2^{7} to $2^{7}-1$

-128 to +127

underflow ↙ ↘ overflow

total values allowed = 256 (2^8)

Short

short a;

2 Bytes

16 bits

-2^{16-1} to $+2^{16-1}-1$

-2^{15} to $+2^{15}-1$

-32768 to +32767

65536 (2^{16})

int

int a;

4 bytes

32 bits

-2^{32-1} to $+2^{32-1}-1$

-2^{31} to $2^{31}-1$

long

long a;

8 bytes

64 bits

-2^{64-1} to $+2^{64-1}-1$

-2^{63} to $+2^{63}-1$

Akash

MON TUE WED THU FRI SAT SUN

Page No.:
Date: __/__/____

→ How integer data stored in Memory?

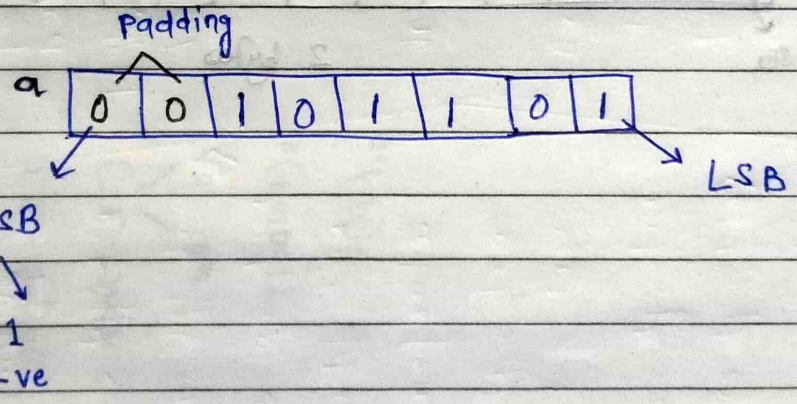
→ byte a = 45;

Base - 2 format

2	45	1
2	22	0
2	11	1
2	5	1
2	2	0
	1	

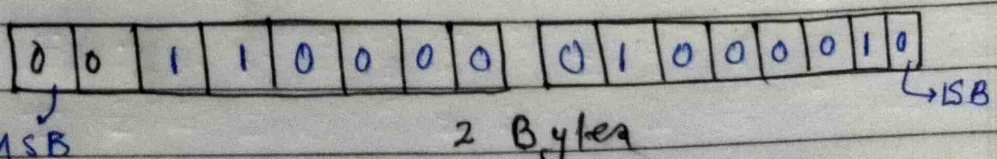
$(45)_{10} = (101101)_2$

1 byte = 8 bit



→ short a = 12354;

$(12354)_{10} = (11000001000010)_2$



Akash

Short a = -12354;

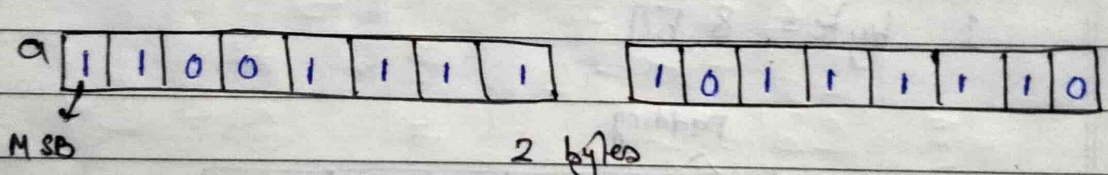
$(12354)_{10} = (0011000000111000010)_2$

To find -ve we have to 2's complement

1's complement = 1100111111011111101

\+ 1

2's complement = 1100111111011111110



Literals :- A Literal is a fixed value written directly in the source code.

Binary (base-2)

```
int a = 0b101101;
S.o.p(a);
```

output = 45

input :

$(101101)_2$

$= (45)_{10}$

Hexadecimal (base-16)

```
int a = 0x45;
S.o.p = (a);
```

output = 69

input : $(45)_{16}$

0100 0101

$(01000101)_2$

$= (69)_{10}$

Octal (Base-8)

```
int a = 045;
S.o.p = (a);
```

output = 37

input (45)

100 101

$(100101)_2$

$= (37)_{10}$

Decimal (Base-10)

```
int a = 45;
S.o.p(a);
```

output = 45

input : $(45)_{10}$

Memory = $(101101)_2$

$= (45)_{10}$

IEEE → Institutes of Electrical & Electronics Engineers.

Page No.: _____
Date: ___/___/_____

MON TUE WED THU FRI SAT SUN

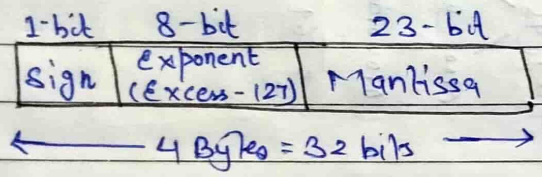
→ Real number data types :-

→ Float :-

float a;

4 bytes.

IEEE - 754 single precision format

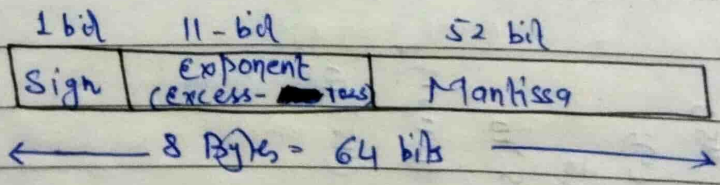


→ Double :-

double a;

8 bytes.

IEEE- 754 double precision format.



MON TUE WED THU FRI SAT SUN

Page No.:
Date: __/__/____

suffix

float a = 24.17f; (IEEE 754 single precision format).

Integral part ← | decimal point | → Fractional part

$(24)_{10} = (11000)_2$

$(.17)_{10} = (.00\ 10101110\ \ldots)_2$

$24.17 = (11000.00100\ \ldots)_2$ {Standard form}

↓

Normalized form [1.significant bits * 2^{exp}]

$1.1100000101110\ \ldots * 2^4$ → exponent

Mantissa.

5236.58

5.23658×10^3

Biased exponent = Exponent + Bias

= 4 + 127

= $(131)_{10}$

= $(10000011)_2$

↓

Exponent

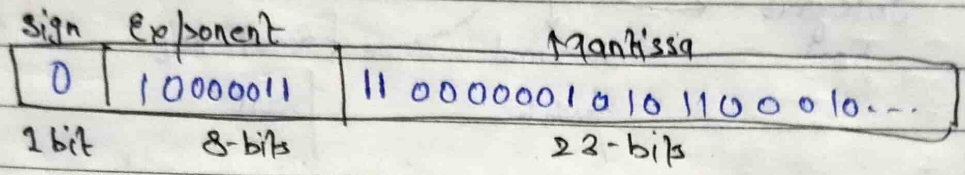
Sign	Exponent [Excess-127]	Mantissa
0	10000011	11000001011100..
1-bit	8-bit	23-bit

Akash

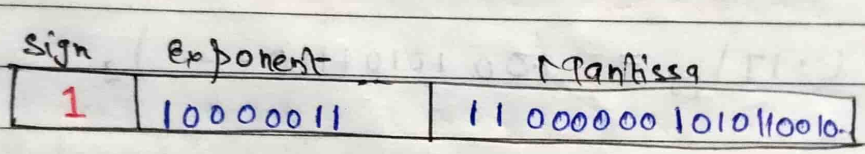
MON TUE WED THU FRI SAT SUN

Page No.: _____
Date: ___/___/_____

float a = 24.17f;

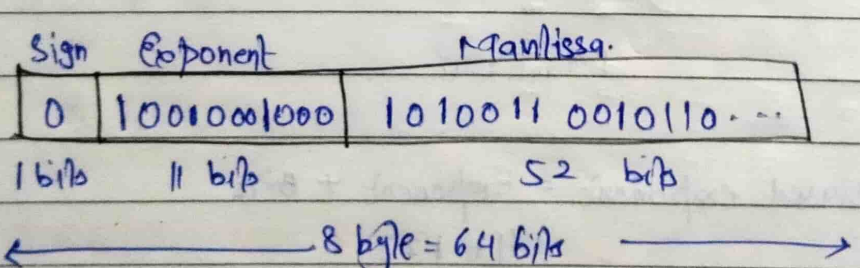


float a = -24.17f;



→ double a = 844.726;

(IEEE 754 double precision format)



Akash

MON TUE WED THU FRI SAT SUN

Page No.:
Date: __/__/____

→ Why Two data type have been provided in JAVA to handle real number data?

→ Floating point literals :-

Standard Notation	Power of 10 notation	Scientific Notation (G)
123.4	1.234×10^2	1.234E2 / 1.234E+2 / 1.234e2 case sensitive
72.543	7.2543×10^1	7.2543E1
400	4.0×10^2	4.0E2 / 4e2
0.354	3.54×10^{-1}	3.54e-1
-0.0003159	3.159×10^{-4}	-3.159e-4

→ Range of real no. data types :-

float :- ± 3.4e38 to ± 3.4e-38

IEEE754

double :- ± 1.7e308 to ± 1.7e-308

Akash

MON TUE WED THU FRI SAT SUN

Page No.: _____
Date: ___/___/_____

45.75 "double" → Java Compiler

double a = 45.75D;
- D → Optional
- Case-insensitive

float a = 45.75; ✗ error : Incompatible type

1.✓ float a = 45.75F
- F → Mandatory
- Case-Insensitive

2.✓ float a = (float) 45.75;
- typecasting

Akash

MON TUE WED THU FRI SAT SUN

Page No.:
Date: __/__/____

→ Character data types :-

Symbol	Binary code
A	0
B	1

2 symbol → 2^1 symbol

↓

1-bit binary code

A	00
B	01
C	10
D	11

4 symbol → 2^2 symbol

↓

2-bit code

- —

— - -

— — - -

16 symbol → 2^4 symbol

↓

4-bit code.

ANSI (American National Standard Institute).

1960's

ASCII (American Standard code for information interchange).

128 symbol → 2^7 symbol

↓

7-bit binary code

0 → NUL (null)	64 → @
1 → SOH (Start of heading)	65 → A
⋮	⋮
⋮	90 → Z
⋮	⋮
32 → SPACE	
⋮	97 → a
⋮	⋮
48 → 0	⋮
⋮	122 → z
57 → 9	⋮
⋮	126 → ~
⋮	127 → DEL
61 → =	
⋮	

ASCII charset

Code point

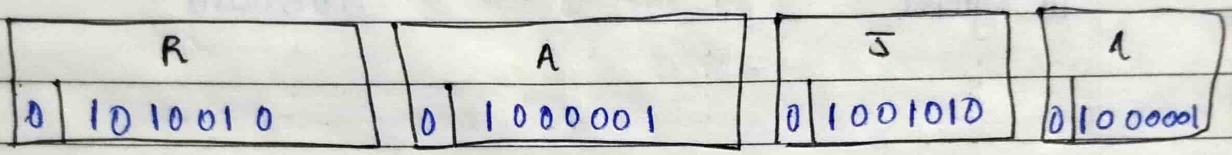
Symbol	Decimal	Binary code
I symbol	0	0000000
II symbol	1	0000001
III symbol	2	0000010
⋮	⋮	⋮
A	65	1000001
B	66	1000010
C	67	1000011
⋮	⋮	⋮
a	97	1100001
b	98	1100010
c	99	1100011
⋮	⋮	⋮
	126	1111110
	127	1111111

C language → ASCII → 7-bit Binary code → Stored as 8-bit (with 1 unused bit).

Data RAJA :- {Real world format}

↓ ASCII

{Binary format}



R → 82 → 1010010

A → 65 → 1000001

J → 74 → 1001010

A → 65 → 1000001

∴ In C, the size of char data type is 1 Byte.

Akash

→ Does Java follow ASCII format like C & C++?

The char data type in Java:

Format: Unicode (UTF-16)
Size: 2 Bytes (16-bits)
Symbol: 65536
Range: 0 to 65535

Symbol	Binary code	Hexadecimal	Decimal
I symbol	0000 0000 0000 0000	0000H	0
II symbol	0000 0000 0000 0001	0001H	1
III symbol	0000 0000 0000 0010	0002H	2
⋮	⋮	⋮	
A	0000 0000 0 1 000001	0041H	65
B	00000000 01000010	0042H	66
C	00000000 01000011	0043H	67
⋮	⋮		
a	00000000 01100001	0061H	97
b	00000000 01100010	0062H	98
c	00000000 01100011	0063H	99
⋮	⋮	⋮	
अ	0000 1001 0000 0101	0905H	2309
आ	0000 1001 0000 0110	0906H	2310
⋮	⋮		
	1111 1111 1111 1110	FFFEH	65534
	1111 1111 1111 1111	FFFFH	65535

→ Three ways to initialize a character literal to a character variable:-

1.) Directly using the character literal enclosed within single quote - 'A'

→ Char ch = 'A';

2.) Using an integeral value -

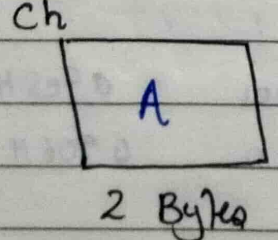
→ Decimal : char ch = 65;
→ Octal : char ch = 0101;
→ hexadecimal : char ch = 0x41;
→ Binary : char ch = 0b1000001;

3.) Using the unicode representation - '\uXXXX'

where XXXX represent 4-digit hexadecimal value of the unicode character.

→ Char ch = '\u0041';

Escape sequence.



→ Escape sequences in Java (control sequences) :-

1.) \t - insert a tab space.
2.) \b - insert a back space.
3.) \n - insert a new line.
4.) \r - insert a carriage return
5.) \f - insert a form feed
6.) \' - insert a single quote
7.) \" - insert a double quote
8.) \\ - insert a backslash
9.) \u - Unicode representation of a character.

Eg:

Expected output: Joe's friend said, "Unicode is a universal charset".

```
System.out.println("joe's friend said," Unicode is a universal charset:");
```

Output :- Compilation error.

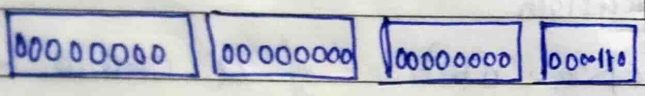
```
System.out.println(Joe's friend said:\" unicode is a universal charset \".');
```

Output :- Joe's friend said, " ~~Universal~~ unicode is a universal charset".

Is there any difference in the way how character data '6' is stored in the memory compared to integer data - 6?

int i = 6;
Base - 2 format

$(6)_{10} = (110)_2$

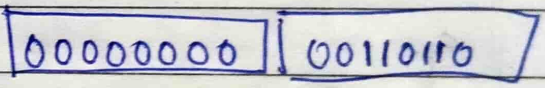


4 bytes

char c = '6';
Unicode format

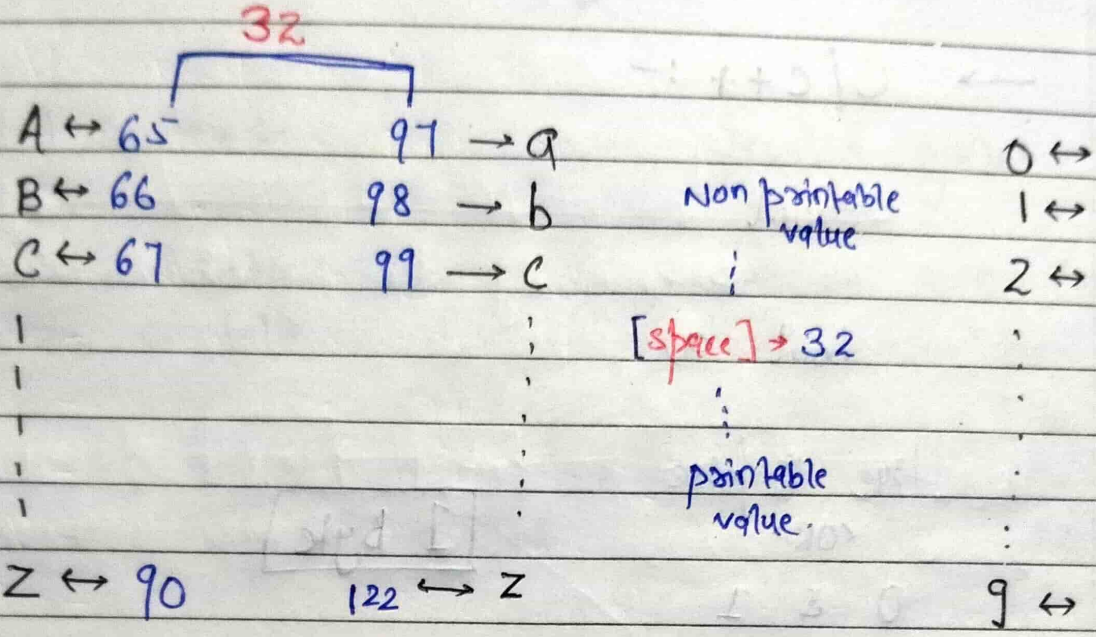
Symbol	Unicode value
6	54

$(54)_{10} = (110110)_2$



2 Bytes

→ Important ASCII / Unicode values.



MON TUE WED THU FRI SAT SUN

Page No.:
Date: / /

→ Yes/NO type of data :-

→ C/C++ :-

bool

bool a;

true & false
<OR>
0 & 1

1 Byte

True ⇒ 1
false ⇒ 0

→ Java :-

boolean

boolean a;

Literals :- true or false

Size : JVM dependent
format :

MON TUE WED THU FRI SAT SUN

Page No.:
Date: __/__/____

In java :-

Valid syntax :-

```
boolean a = true;
boolean a = false;
```

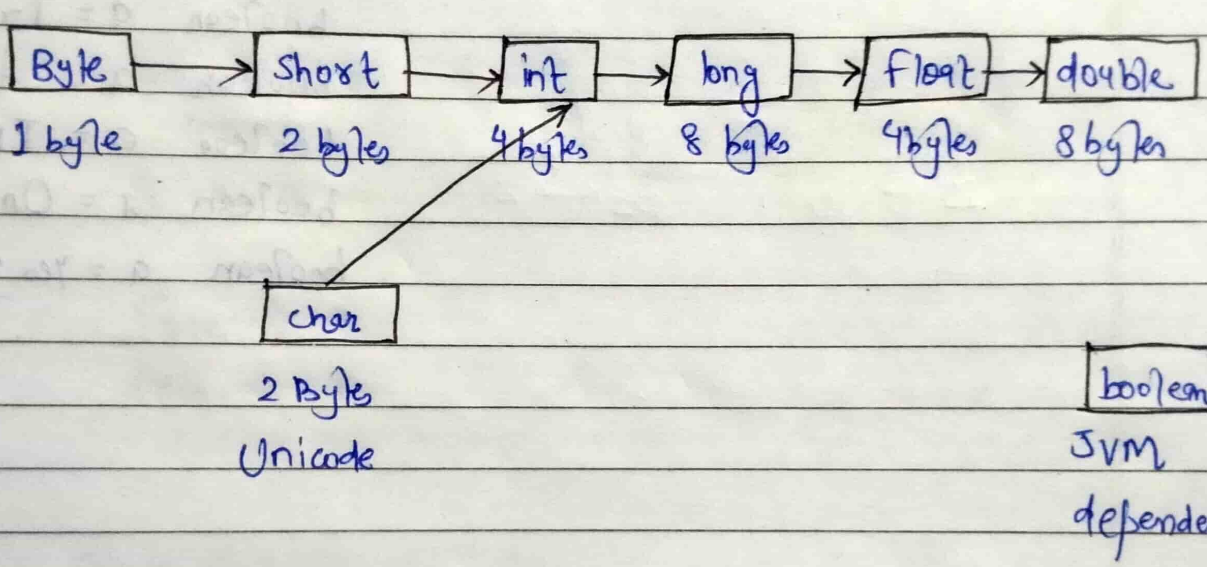
Invalid syntax :-

```
boolean a = 0; (possible in C).
boolean a = "true";
boolean a = True;
boolean a = 'true';
boolean a = TRUE;
boolean a = On;
boolean a = yes;
```

Type casting / Type Conversion

Implicit type casting / Numeric promotion / widening / upcasting

Automatically done by the compiler



Explicit type casting / Downcasting

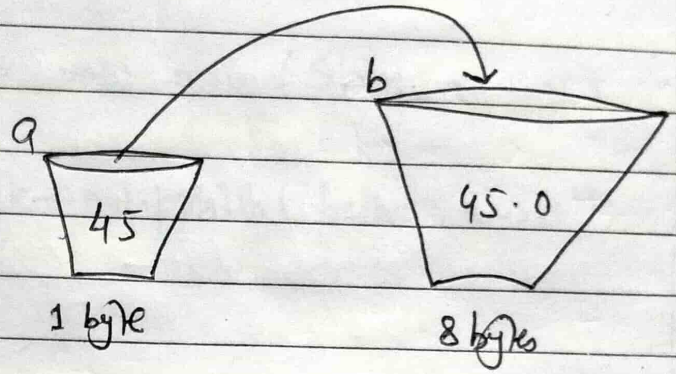
Manually done by the programmer

Akash

→ Implicit type casting / Numeric promotion / widening / Upcasting

Eg;

```
byte a = 45;
double b;
b = a;

S.o.p (a);
S.o.p (b);
```



output :-

45
45.0

Advantages :- No loss of information (data or precision).

→ Explicit type casting / Downcasting :-

Eg;

```
double a = 45.5;
byte b;
b = (byte) a;

S.o.p (a);
S.o.p (b);
```

DisAdvantages :- Possible loss of information (data or precision).

Output :- 45.5
45.

Akash